Manual Técnico

Admisiones USAC · Proyecto de Bases de Datos NoSQL · 30 de octubre de 2026

Sistema de análisis de pruebas específicas de aspirantes a Ingeniería USAC.

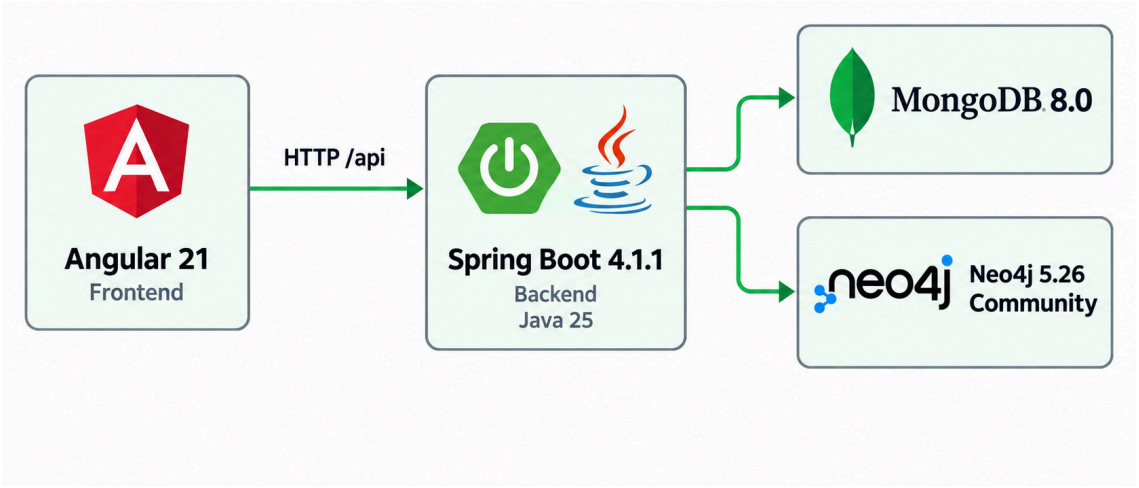
1. Objetivo y alcance

La solución importa archivos CSV de pruebas específicas, valida y transforma sus registros, los almacena en MongoDB y Neo4j, y ofrece estadísticas, búsqueda de aspirantes y exploración de relaciones. Se utiliza el conjunto oficial Datos Abiertos de Ingeniería USAC. Este manual documenta la arquitectura, instalación, modelos, ETL, consultas y criterios de verificación.

Fuente: https://datosabiertos.ingenieria.usac.edu.gt/en/dataset/pruebas-especificas

2. Arquitectura y tecnologías

Arquitectura de tres niveles:



- El backend centraliza validaciones, consultas, conexiones y errores; el navegador nunca accede directamente a las bases ni a sus credenciales.
- El frontend usa TypeScript, componentes Angular y gráficas de barras; vis-network presenta los nodos y aristas del grafo.
- Docker Compose administra las bases y sus volúmenes persistentes.
- Se prefieren MongoDB para documentos y agregaciones, y Neo4j para recorridos entre entidades.

Diagrama de Arquitectura

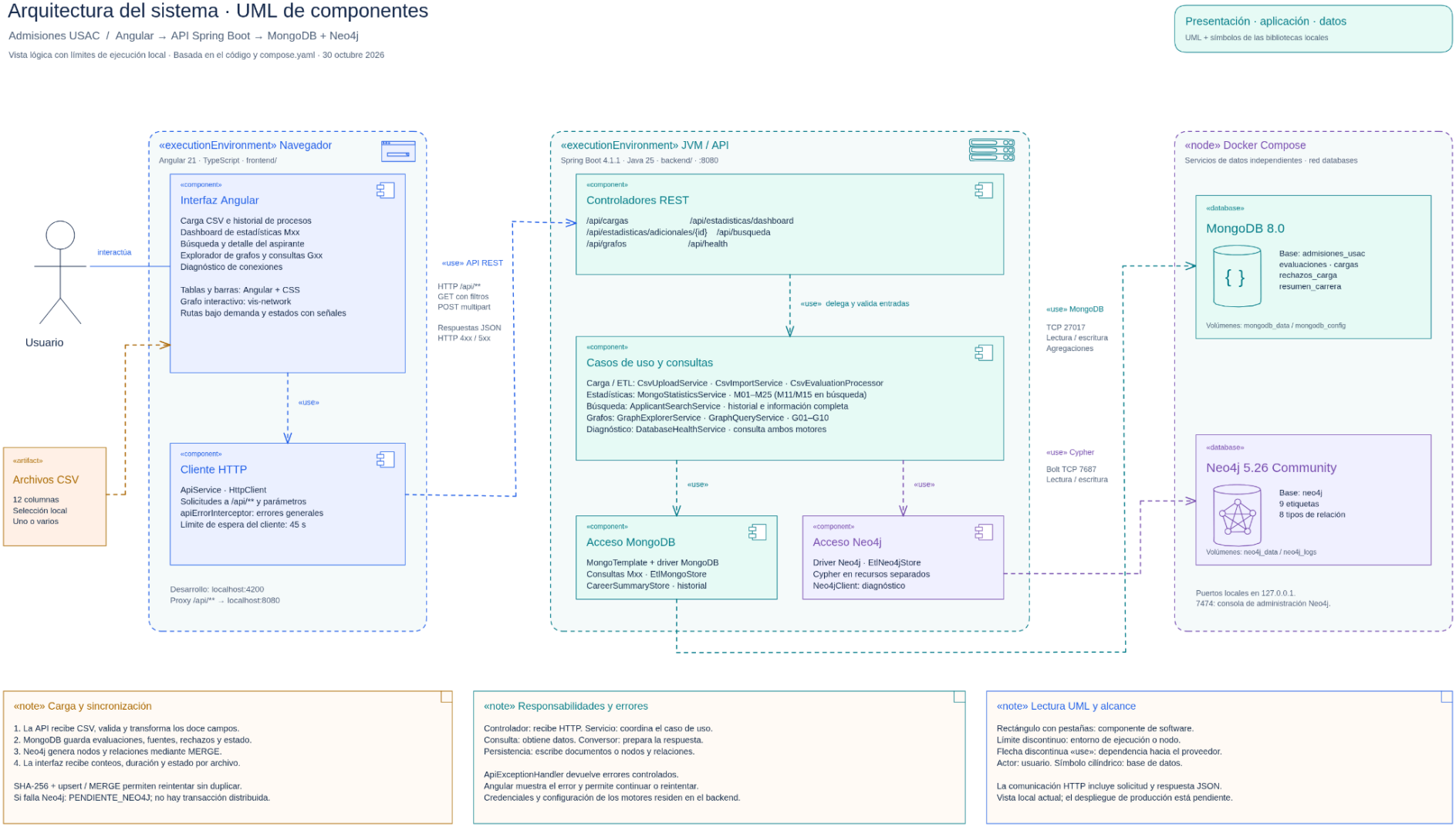
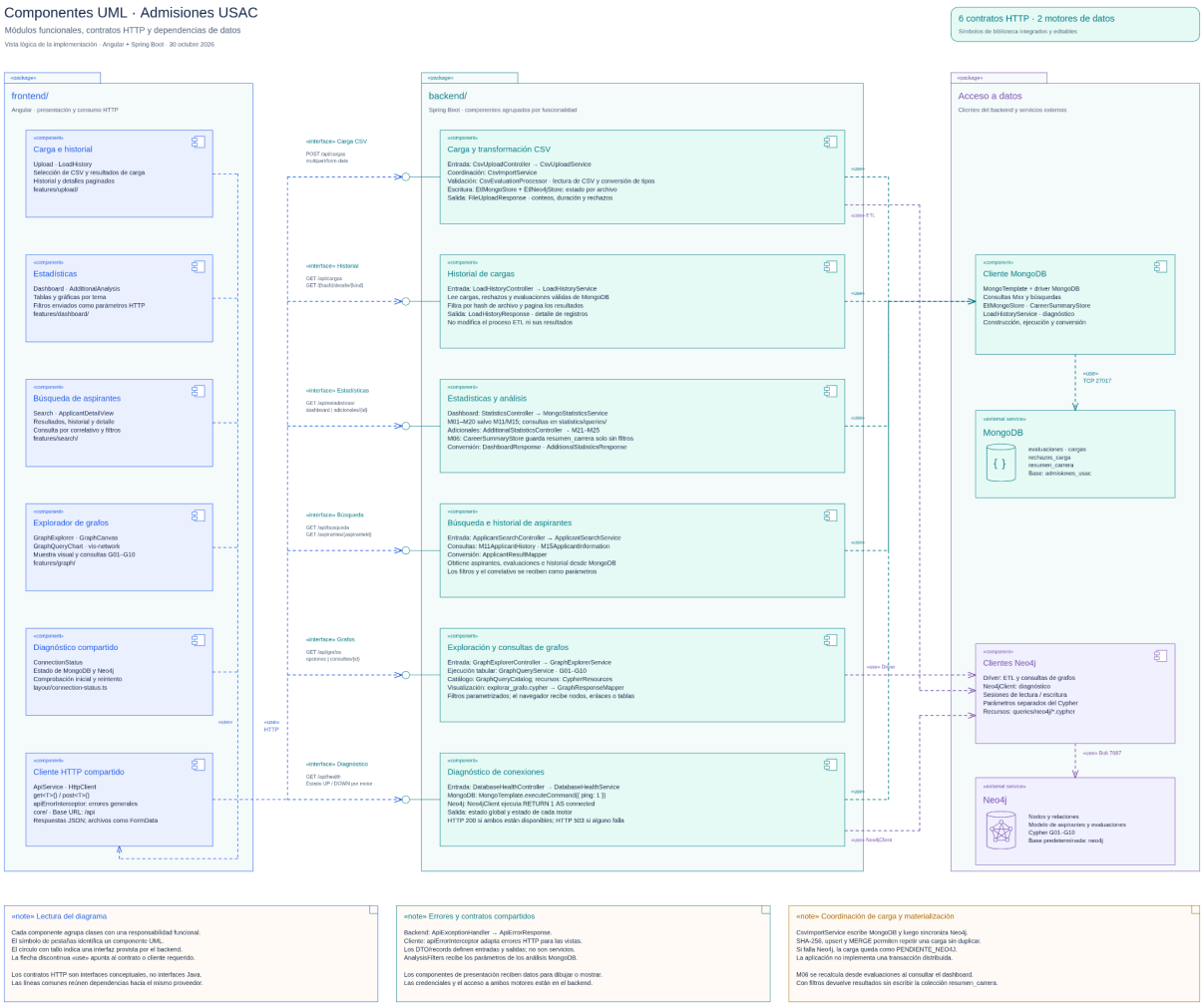


Diagrama de Componentes



3. Requisitos e instalación local

Requisitos:
Git, JDK 25, Docker Engine con Compose v2, Node.js 20.19+/22.12+/24.x compatible con package.json y npm 10 o superior. Maven se ejecuta mediante el wrapper del repositorio. Python 3 es opcional para validadores.

1) Clonar la rama de referencia y crear la configuración sin guardar contraseñas en Git:

```
git clone -b tmp/docs https://github.com/BrigidoAlvarado/NoSQL-BD2.git cd NoSQL-BD2 && cp .env.example .env
```

2) Editar .env y definir:
   a) MONGODB_USERNAME
   b) MONGODB_PASSWORD

c) NEO4J_PASSWORD (mínimo ocho caracteres para Neo4j).

Mantener puertos y nombres de bases predeterminados, salvo necesidad local.

3) Levantar los motores y aplicar los scripts de índices/restricciones antes de la primera carga:

```
docker compose up -d --wait

docker compose exec -T mongodb sh -c 'mongosh --quiet --username
"$MONGO_INITDB_ROOT_USERNAME" --password "$MONGO_INITDB_ROOT_PASSWORD"
--authenticationDatabase admin admisiones_usac' <
backend/src/main/resources/db/mongodb/01_indices.js

cat backend/src/main/resources/db/neo4j/01_esquema.cypher | docker compose
exec -T neo4j sh -c 'cypher-shell --non-interactive --username neo4j
--password "${NEO4J_AUTH#*/}" --database neo4j'
```

4) Iniciar backend y frontend en terminales diferentes:

```
cd backend && ./mvnw spring-boot:run
cd frontend && npm ci && npm start
```

5) Abrir http://localhost:4200 y comprobar http://localhost:8080/api/health.

   Estado UP en ambas bases confirma conexión, no necesariamente la existencia de registros.

   Puertos locales: Angular 4200; API 8080; MongoDB 27017; Neo4j Browser 7474 y Bolt 7687.

Los volúmenes Docker conservan datos al detener servicios. No usar docker compose down --volumes para un reinicio de prueba.

4. Diseño de almacenamiento

El sistema utiliza MongoDB y Neo4j de manera complementaria. MongoDB almacena los registros de evaluación y permite realizar consultas estadísticas, mientras que Neo4j representa las relaciones entre aspirantes, evaluaciones y sus diferentes características. Esta separación aprovecha las capacidades de cada modelo NoSQL.

4.1. Modelo MongoDB

Se diseñó un modelo documental centrado en las evaluaciones, donde cada documento representa una evaluación única. Esta decisión permite almacenar los datos originales y procesados de forma estructurada, facilitando las consultas analíticas mediante *Aggregation Pipelines.*

El modelo utiliza cuatro colecciones principales:

Evaluaciones: almacena los registros procesados y sus datos académicos.

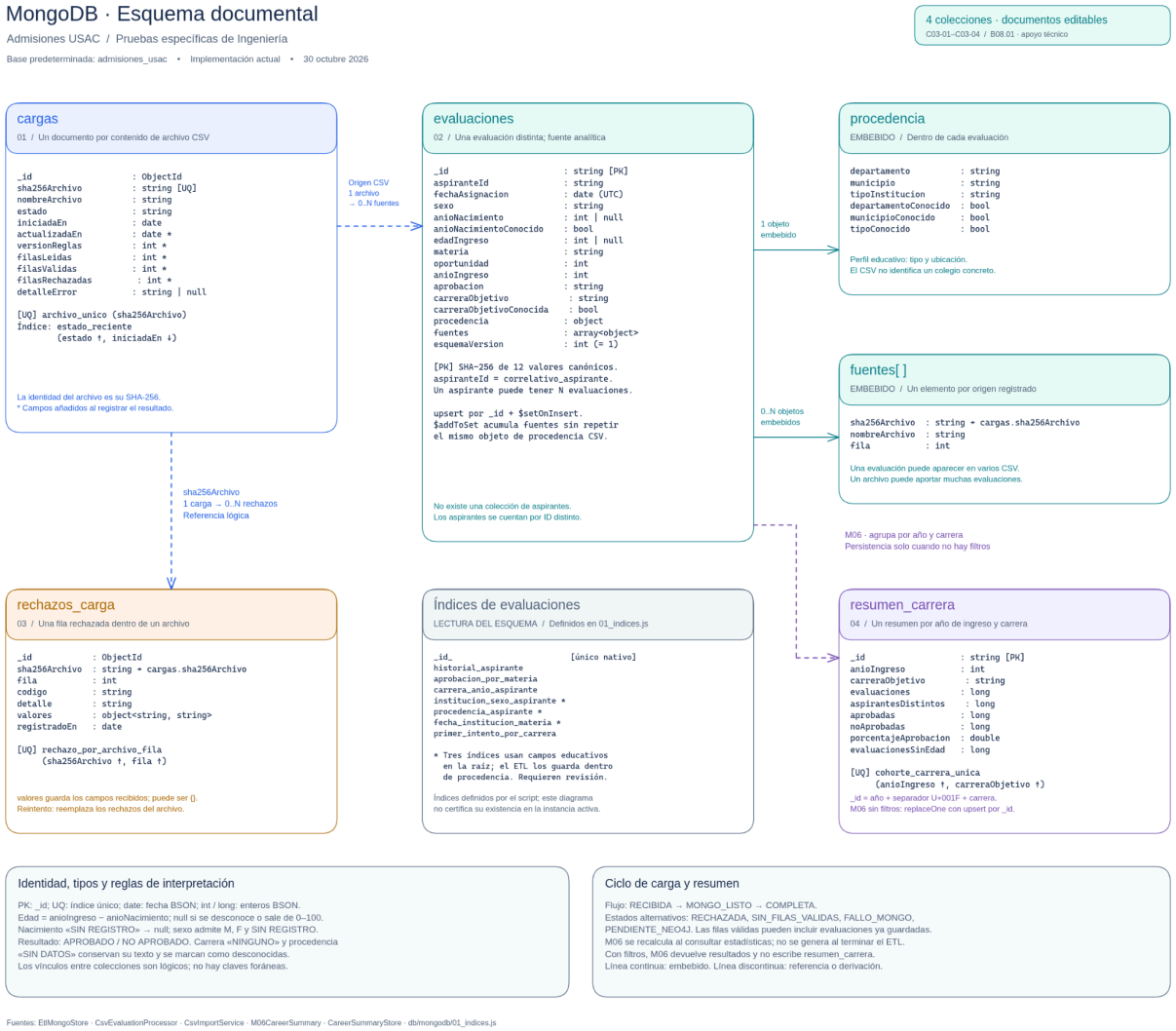
cargas: registra los archivos importados y el estado de su procesamiento.

rechazos_carga: conserva los registros inválidos y las causas de rechazo.

-**resumen_carrera**: almacena estadísticas consolidadas por carrera y año de ingreso.

Se decidió almacenar las evaluaciones individualmente en lugar de agruparlas dentro de un documento por aspirante, porque una misma persona puede realizar múltiples evaluaciones, incluso en diferentes años. Esta estructura evita documentos de crecimiento indefinido y facilita los filtros, agrupaciones y cálculos estadísticos solicitados.

Cada evaluación utiliza un identificador SHA-256 generado a partir de sus doce campos canónicos, lo que permite evitar duplicados exactos sin eliminar evaluaciones legítimas. El campo `aspiranteId` conserva el identificador anónimo para reconstruir su historial. Asimismo, se definieron índices para optimizar las búsquedas y consultas más frecuentes.



4.2. Modelo Neo4j

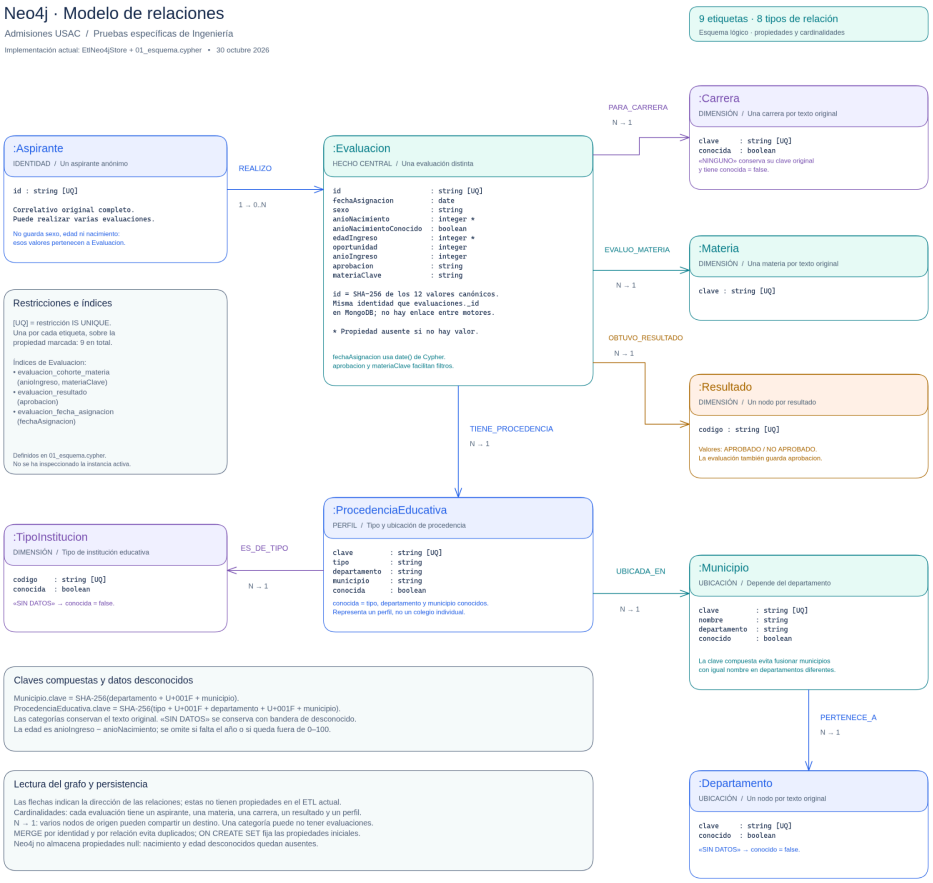
Se implementó un modelo de grafos para representar explícitamente las conexiones entre los elementos del proceso de evaluación. Este enfoque facilita consultas que requieren recorrer varias relaciones, como identificar carreras compartidas entre aspirantes, analizar intentos de evaluación o relacionar resultados con la procedencia educativa.

El modelo está compuesto por nueve tipos de nodos: Aspirante, Evaluacion, Materia, Carrera, Resultado, ProcedenciaEducativa, TipoInstitucion, Municipio y Departamento.

Las relaciones conectan a cada aspirante con sus evaluaciones y, a partir de ellas, con sus materias, carreras, resultados y procedencia. Esta estructura permite reutilizar entidades comunes en lugar de crear nodos repetidos por cada registro.

Para garantizar la consistencia, se utilizan restricciones de unicidad y operaciones `MERGE`, que evitan duplicar nodos y relaciones durante cargas repetidas. Además, cada evaluación conserva el mismo identificador utilizado en MongoDB, permitiendo relacionar los registros almacenados en ambos motores.

Se creó *ProcedenciaEducativa* para representar el tipo y la ubicación de la institución de origen, ya que el conjunto de datos no proporciona nombres ni identificadores de establecimientos específicos. De esta forma, el modelo representa únicamente información respaldada por el origen.



5. Extracción, transformación y carga (ETL)

Se implementó un proceso ETL para garantizar que la información proveniente de los archivos CSV sea validada y transformada antes de almacenarse en MongoDB y Neo4j. Esto es necesario porque los archivos originales contienen valores desconocidos, nombres truncados y registros duplicados que podrían afectar los resultados estadísticos.

El procesamiento comprende las siguientes etapas:

1. **Extracción**: lectura de uno o varios archivos CSV en UTF-8, identificando sus doce campos mediante los encabezados originales y sus variantes conocidas.
2. **Transformación**: validación de datos obligatorios, conversión de fechas y números, normalización de valores definidos y tratamiento de información faltante.
3. **Carga documental**: almacenamiento de evaluaciones válidas en MongoDB mediante operaciones *upsert*, evitando duplicados exactos.
4. **Carga de grafos**: creación o reutilización de nodos y relaciones en Neo4j mediante operaciones `MERGE`.
5. **Registro de resultados**: almacenamiento del estado de cada carga, cantidades procesadas, rechazos, duración y causas de error.

5.1. Reglas de validación y transformación

Las reglas se definieron buscando **conservar la mayor cantidad posible de información sin alterar su significado original**.

Por ejemplo, **SIN REGISTRO** en el año de nacimiento no provoca el rechazo de una evaluación; se conserva el registro y se representa la edad desconocida mediante *null*. De igual forma, **NINGUNO** en carrera y **SIN DATOS** en procedencia se mantienen como valores desconocidos para evitar interpretaciones incorrectas.

La edad se calcula restando el año de nacimiento al año de ingreso, y se considera desconocida cuando falta información o el resultado queda fuera del rango de 0 a 100 años. Por otra parte, los registros con fechas inválidas, oportunidades no positivas o campos obligatorios incorrectos se rechazan, registrando su línea y causa.

También se conservan las categorías originales de carreras y ubicaciones, ya que corregir automáticamente nombres truncados podría combinar entidades diferentes y alterar las estadísticas.

5.2. Control de duplicados y consistencia

Para evitar duplicaciones, se utilizan dos identificadores SHA-256: uno para reconocer el contenido completo del archivo y otro para identificar cada evaluación mediante sus doce valores canónicos.

Esta estrategia permite reconocer cargas repetidas y conservar evaluaciones diferentes realizadas por un mismo aspirante. También mantiene la trazabilidad mediante referencias al archivo y a la fila de origen.

Debido a que MongoDB y Neo4j funcionan como motores independientes, no existe una transacción distribuida entre ambos. Por ello, el sistema registra estados como *COMPLETA*, *FALLO_MONGO* y *PENDIENTE_NEO4J*.

Si MongoDB almacena correctamente los registros, pero falla Neo4j, la carga queda pendiente de sincronización. Al reenviar el mismo archivo, el sistema puede reintentar el proceso sin duplicar las evaluaciones almacenadas.

Esta estrategia permite recuperar cargas parcialmente completadas, conservar la integridad de la información y mantener un seguimiento de los errores, cumpliendo los requisitos de validación, transformación y almacenamiento en ambos motores.

Finalmente, las consultas distinguen entre evaluaciones y aspirantes únicos. Esta diferencia es fundamental porque un aspirante puede aparecer en múltiples registros y contar cada evaluación como una persona produciría estadísticas incorrectas.

6. Consultas analíticas MongoDB

La aplicación implementa las veinte consultas exigidas (M01–M20), con código independiente por identificador. M01–M10, M12–M14 y M16–M20 se consultan desde el dashboard; M11 y M15 desde el detalle de un aspirante. Filtros opcionales se aplican antes de agrupar y recalculan el subconjunto.

M01 — Aspirantes distintos por tipo de institución.
M02 — Evaluaciones aprobadas por materia.
M03 — Aspirantes con aprobaciones por carrera y año de ingreso.
M04 — Porcentaje de evaluaciones aprobadas por materia.
M05 — Edad promedio de aspirantes por carrera, con edad válida.
M06 — Resumen por carrera y cohorte; materializa resumen_carrera sin filtros.
M07 — Edad promedio de aprobados por carrera y tipo de institución.
M08 — Aspirantes aprobados por municipio, departamento y carrera.
M09 — Evaluaciones por mes y materia en instituciones públicas.
M10 — Cinco carreras con más aspirantes de 16 a 18 años.
M11 — Historial de materias, oportunidades y resultados del aspirante.
M12 — Aspirantes distintos por sexo y tipo de institución.
M13 — Tasa de aprobación de evaluaciones por edad válida.
M14 — Promedio de oportunidades distintas por materia y aspirante/cohorte.
M15 — Información completa disponible del correlativo anónimo.
M16 — Aspirantes no aprobados en la oportunidad 1 por carrera.
M17 — Cinco municipios conocidos con más aspirantes.
M18 — Tasa de aprobación por tipo de institución.
M19 — Carrera más demandada por departamento (incluye empates).
M20 — Evolución anual de aprobadas, no aprobadas y porcentaje.

Consultas complementarias M21–M25:

M11 — Historial de materias, oportunidades y resultados del aspirante.
M12 — Aspirantes distintos por sexo y tipo de institución.
M13 — Tasa de aprobación de evaluaciones por edad válida.
M14 — Promedio de oportunidades distintas por materia y aspirante/cohorte.
M15 — Información completa disponible del correlativo anónimo.
M16 — Aspirantes no aprobados en la oportunidad 1 por carrera.
M17 — Cinco municipios conocidos con más aspirantes.
M18 — Tasa de aprobación por tipo de institución.
M19 — Carrera más demandada por departamento (incluye empates).
M20 — Evolución anual de aprobadas, no aprobadas y porcentaje.

Los resultados se presentan en tablas y gráficas; la gráfica limita a ocho grupos, mientras la tabla mantiene todas las filas.

7. Consultas Cypher y exploración de grafos

Neo4j implementa diez consultas, más de las cinco mínimas. Cada archivo Gxx_*.cypher contiene la consulta ejecutable: usa MATCH sobre nodos/relaciones, propiedades, recorridos, agrupaciones y filtros. La API parametriza valores; no incorpora expresiones Cypher enviadas por el usuario.
G01 — Demanda de evaluaciones por carrera.
G02 — Resultados de evaluaciones por materia.
G03 — Aspirantes aprobados por carrera y departamento de procedencia.
G04 — Pares de carreras presentes en evaluaciones de un mismo aspirante.
G05 — Resultados por cohorte, carrera y materia.
G06 — Resultados por tipo de institución educativa y materia.
G07 — Reintentos por aspirante, materia y cohorte.
G08 — Demanda por departamento, municipio y carrera
G09 — Evaluaciones por sexo declarado, carrera, materia y resultado.
G10 — Aspirantes con aprobaciones y no aprobaciones para una misma carrera.

Ruta común: backend/src/main/resources/queries/neo4j/. GraphQueryCatalog registra recursos y GraphQueryService los ejecuta. Se aplican filtros a grupos ya calculados (conservan conteos); los filtros del grafo visual operan por separado sobre la muestra. Esta distinción evita interpretar ambas vistas como idénticas.
El explorador usa explorar_grafo.cypher, permite filtrar por año, carrera, resultado y departamento, y muestra una muestra de 5 a 40 evaluaciones. La cantidad de nodos no equivale al total poblacional de las consultas G01–G10.

8. API y funciones principales

GET /api/health — diagnóstico independiente de MongoDB y Neo4j (200/503).
POST /api/cargas — carga multipart de files (30 MB/archivo, 100 MB/solicitud).
GET /api/cargas?page=0 — historial; detalle
/api/cargas/{hash}/detalle/{validos|rechazos}?page=0.
GET /api/estadisticas/dashboard — 18 resultados MongoDB con filtros opcionales.
GET /api/estadisticas/adicionales/{id} — análisis M21–M25.

GET /api/busqueda y /api/busqueda/aspirantes/{correlativo} — búsqueda e historial M11/M15.
GET /api/grafos y /api/grafos/opciones — muestra visual y opciones de filtros.
GET /api/grafos/consultas/{id} — resultados analíticos G01–G10.
El frontend integra Carga de datos, Estadísticas, Búsqueda y Explorador de grafos. El backend devuelve mensajes controlados; Angular los presenta en español y mantiene acciones de reintento.

9. Pruebas y resultados verificados

Comandos de revisión reproducibles (requieren JDK 25 y, para integraciones, ambos motores activos y sus esquemas aplicados):

```
cd backend && ./mvnw verify
./mvnw -Detl.integration=true -Dsearch.integration=true

-Dstatistics.integration=true -Dgraph.integration=true

-Dadditional.integration=true -Dfilters.integration=true verify

cd frontend && npm test && npm run build

python3 -m unittest discover -s scripts -p 'test_*.py'
```

Evidencia documentada al 29/10/2026: pruebas unitarias e integraciones de ETL, estadísticas, búsquedas y Cypher; pruebas Angular y compilación correctas. La instantánea de las cohortes cargadas 2015 y 2022 registra 44 505 evaluaciones distintas, 12 528 aspirantes únicos y 6 407 aprobadas (14,40 %).

10. Seguridad, limitaciones y conclusiones

10.1 Seguridad

Las credenciales se definen en .env excluido de Git y los puertos de las bases se publican en 127.0.0.1 para desarrollo. La API valida parámetros, evita exponer trazas y permite recuperar cargas parciales. Antes de un despliegue público deben revisarse las alertas de dependencias, configuración de producción y pruebas operativas.

10.2 Limitaciones

El origen contiene datos incompletos y categorías truncadas: los promedios de edad solo incluyen observaciones válidas; NINGUNO no se interpreta como carrera real; la procedencia es de la institución declarada, no residencia del aspirante. Los análisis describen asociaciones, no causalidad.

10.3 Conclusiones

MongoDB facilita estadísticas y trazabilidad por evaluación; Neo4j facilita recorridos y relaciones. Su uso conjunto satisface el objetivo del proyecto, con el costo de mantener

sincronización y consistencia entre dos motores. La idempotencia de la carga permite reintentos sin descartar oportunidades legítimas.